

Αναφορά πάνω στην 2η εργασία

#####

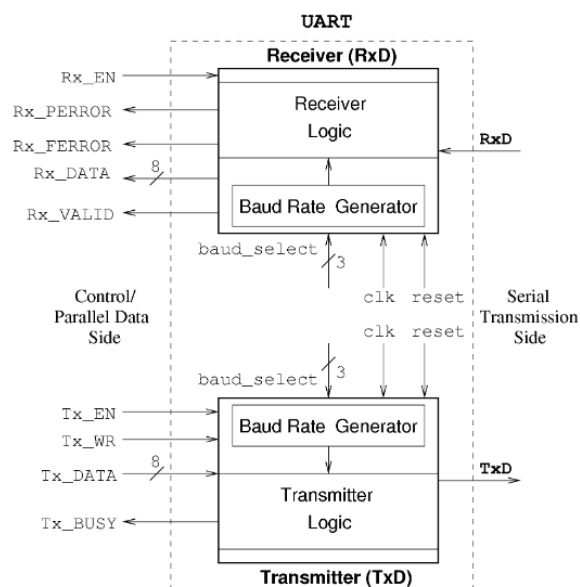
AEM: ####

10/11/2024

- **Περίληψη:**

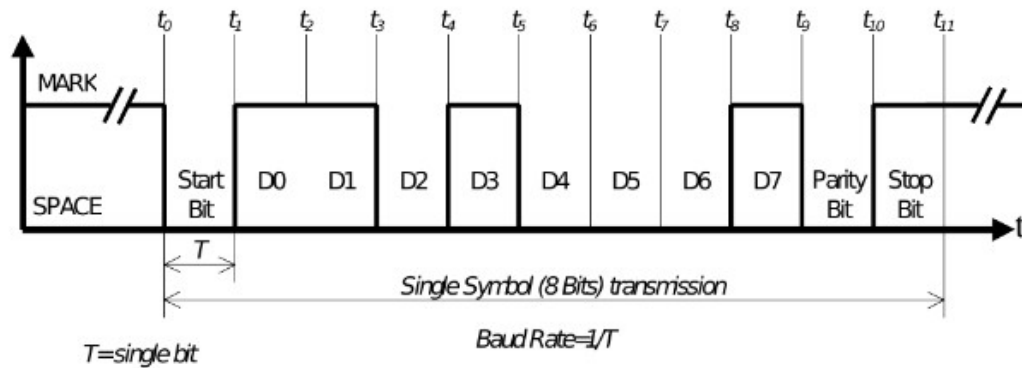
Στην αναφορά αυτή παρουσιάζεται η υλοποίηση ενός ζευγαριού αποστολέα και δέκτη, που χρησιμοποιούν το πρωτόκολλο UART για τη σειριακή μεταφορά δεδομένων, η εξήγηση και λεπτομέρειες αυτής, καθώς και λεπτομέρειες σχετικά με την επαλήθευση σωστής λειτουργίας τους. Συγκεκριμένα παρουσιάζεται η υλοποίηση μονάδας ελέγχου baud rate για τον συγχρονισμό τους, την υλοποίηση των fsm που στηρίζουν τη λογική τους και περιφεριακά κομμάτια της που επιτρέπει τη σωστή λειτουργία τους.

- **Εισαγωγή:**



Παράδειγμα γενικής μορφής αποστολέα και δέκτη

Όπως αναφέρθηκε προηγουμένως, κύριος στόχος της εργασίας είναι η υλοποίηση ενός UART transmitter και ενός UART receiver, και η ασύγχρονη επικοινωνία μεταξύ τους. Για την επικοινωνία ενός συμβόλου ο αποστολέας θα χρησιμοποιήσει το παρακάτω χρονοδιάγραμμα για την αποστολή ενός συμβόλου (8-bit).



Χρονοδιάγραμμα αποστολής ενός συμβόλου

Το start bit σηματοδοτεί στον δέκτη την έναρξη αποστολής δεδομένων και τον επιτρέπει να ευθυγραμμιστεί, ενώ το stop bit σηματοδοτεί το τέλος αποστολής δεδομένων. Το parity bit αταδεικνύει αν το σύμβολο που μεταβιβάστηκε περιέχει άρτιο ή περιττό αριθμό λογικών άσων, και χρησιμοποιείται ως έλεγχος ορθότητας των δεδομένων από τον δέκτη.

Επειδή η προσυμφωνία και η επιλογή μίας κοινής ταχύτητας δεν είναι μέρος του πρωτοκόλλου επικοινωνίας, κάθε μονάδα περιλαμβάνει τον δικό της προγραμματιζόμενο ελεγκτή Baud Rate, ο οποίος θέτει τον ρυθμό μετάδοσης πριν την έναρξη της επικοινωνίας, για τον σωστό συγχρονισμό αποστολής και λήψης δεδομένων. Υποστηρίζονται οκτώ ταχύτητες επικοινωνίας UART, οι οποίες φαίνονται στον παρακάτω πίνακα

BAUD_SEL	Baud Rate (bits/sec)
0 0 0	300
0 0 1	1200
0 1 0	4800
0 1 1	9600
1 0 0	19200
1 0 1	38400
1 1 0	57600
1 1 1	115200

Πίνακας υποστηριζόμενων ταχυτήτων αποστολής σε bits/s

Παραπάνω λεπτομέρειες σχετικά με την υλοποίηση και εισόδους/εξόδους φαίνονται παρακάτω.

- **Μέρος 1ο: Ελεγκτής Baud Rate**

Σε αυτό το κομμάτι της εργασίας υλοποιούμε έναν ελεγκτή baud rate, το οποίο δημιουργεί ένα sampling σήμα ανα δεδομένο χρονικό διάστημα, η διάρκεια του οποίου μπορεί να πάρει μια απο οκτώ προκαθορισμένες τιμές.

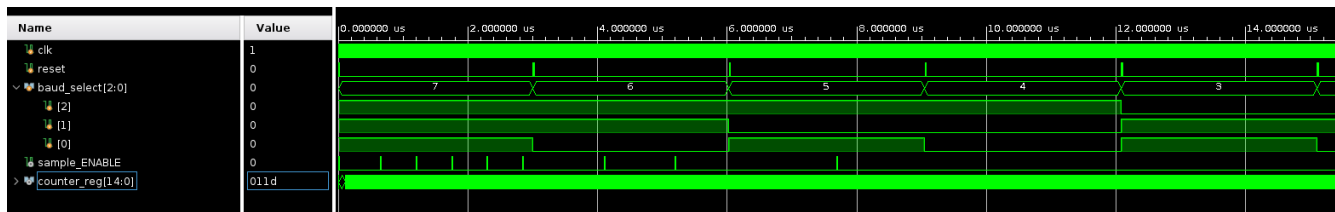
Σαν εισόδους χρησιμοποιεί το baud select, που ρυθμίζει ποιά απο τις οκτώ διάρκειες θα επιλαγεί για την έξοδο όταν είναι ανοιχτή, και η είσοδος enable που θα σταματάει τη δημιουργία σημάτων sampling όταν δεν είναι ενεργό. Αποτελείται απο δύο always block, το ένα ακολουθιακής λογικής που χρησιμοποιεί έναν μετρητή για να υπολογίσει πότε πρέπει να ανεβεί το sampling σήμα, ανάλογα με την μέγιστη τιμή που μπορεί να πάρει, και ένα δεύτερο που έχει στη λίστα ευαισθησίας του μόνο το baud select, και ανάλογα με τη τιμή του θα θέσει τη μέγιστη τιμη που μπορεί να πάρει ο μετρητής. Το σήμα sampling ανεβεί για κάθε κύκλο που η τιμή του μετρητή γίνει ίση με την μέγιστη τιμή του. Όταν η είσοδος enable δέν είναι ανοιχτή, ο μετρητής μηδενίζεται. Αξίζει να σημειωθεί πως δεν περιμένουμε αλλαγή του baud select χωρίς reset, οπότε δεν έχουμε πάρει μέτρα να αντιμετωπίσουμε τη περίπτωση και το θεωρούμε λάθος του χρήστη.

Για να μετρήσουμε σωστά την ποσότητα baud rate με το ρολόι, θα χρησιμοποιήσουμε τον τύπο $\text{NumOfClks} = 1/(16 \cdot \text{baudrate}) \cdot \text{clk_period}$. Για παράδειγμα, για baud_rate = 9600 (που θα έχουμε για baud_sel = 4), ο αριθμός των κύκλων του ρολογιού που θέλουμε για να το μετρήσουμε, δηλαδή και η μέγιστη τιμή του μετρητή είναι $\text{NumOfClks} = 1/(16 \cdot 9600 \cdot 10^{-8}) = 651.0416666$, το οποίο θα στρογγυλοποιήσουμε σε 614. Τα αποτελέσματα των πράξεων για κάθε διαφορετικό baud rate, καθώς και τα σφάλματα αυτών φαίνονται στον παρακάτω πίνακα

Baud Select	Baud Rate (Bits/s)	Clock Cycles	Rounding error
000	300	20833	0.3333...
001	1200	5208	0.3333...
010	4800	1303	0.1666...
011	9600	651	0.041666...
100	19200	326	0.4791666...
101	38400	163	0.23958333...
110	57600	109	0.4930555...
111	115200	54	0.25347222...

PartA Baud Rate Table

Για την επαλήθευση απλώς ελέγχουμε για κάθε διαφορετικό baud select, αν ο μετρητής λειτουργεί σωστά και αν τα σήματα sampling βγαίνουν σωστά.



PartA Baud Controller Post-Synthesis Functional Simulation

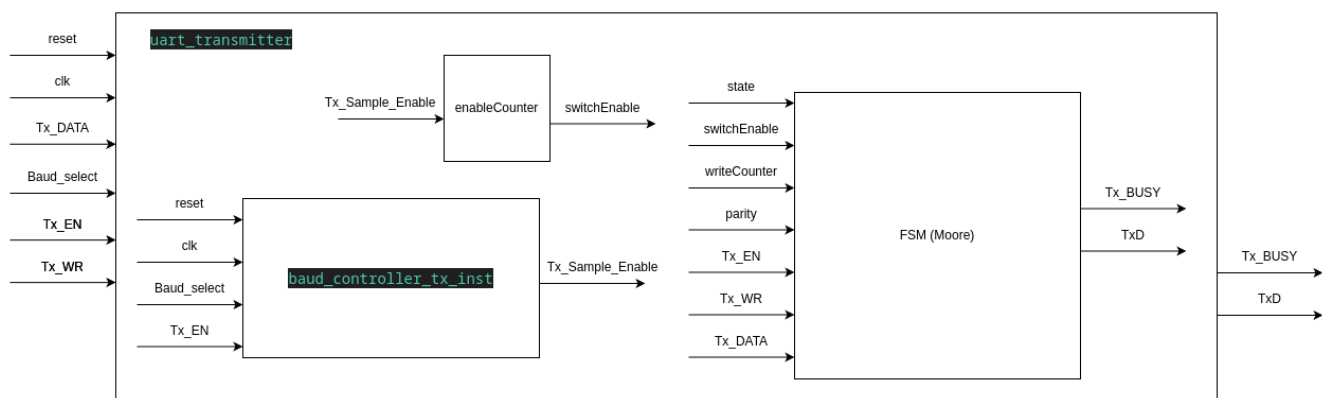
Όπως φαίνεται και στις κυματομορφές, μειώνουμε το baud select σήμα (αρχικά είναι το μέγιστο) “πατώντας” ταυτόχρονα και το reset. Η συχνότητα εμφάνισης του sample_ENABLE μειώνεται όπως θα περιμέναμε

Λόγω ασάφειας στην εκφώνηση το enable σήμα προστέθηκε μετά την πρώτη εξέταση σαν είσοδος, οπότε δέν καλύπτεται απο το πλαίσιο δοκιμής. Η ορθή του λειτουργία όμως επιβεβαιώνεται απο την ορθή λειτουργία του αποστολέα και του δέκτη, στα οποία θα ελέγξουμε τα δικά τους enable σήματα, άρα και αυτό του baud controller κατα συνέπεια.

• Μέρος 2ο: Υλοποίηση UART Αποστολέα (transmitter)

Το κύκλωμα που φτιάξαμε στο 2ο κομμάτι της εργασίας υλοποιεί έναν αποστολέα δεδομένων ο οποίος συμμορφώνεται στο πρωτόκολλο UART. Συγκεκριμένα, παίρνει σαν εισόδους του τον “χαρακτήρα” που θέλουμε να μεταδώσουμε (ένα δεδομένο 8-bit), το baud select που θα ρυθμίσει τον ρυθμό με τον οποίο θα στέλνουμε δεδομένα, καθώς και τα σήματα ελέγχου Tx_EN(enable) και Tx_WR(write), τα οποία ρυθμίζουν αν ο αποστολέας είναι ενεργός και αν του έχει δοθεί σήμα για να σταλεί αντίστοιχα, ενώ σαν εξόδους βγάζει το TxD (το bit που στέλνουμε), και το Tx_BUSY, το οποίο είναι ενεργό όταν στέλνονται δεδομένα.

Παρακάτω φαίνεται ένα flow diagramm για τον αποστολέα. Λεπτομέρειες στην υλοποίηση εξηγούνται αναλυτικά μετά το διάγραμμα.



PartB Dataflow Diagramm

Αρχικά, για να ρυθμίσουμε τον ρυθμό αποστολής των δεδομένων και τον σωστό συγχρονισμό με τον δέκτη, θα χρησιμοποιήσουμε μία μονάδα baud_controller που υλοποιήσαμε προηγουμένως.

Για την αποστολή των δεδομένων θα χρησιμοποιήσουμε μία μηχανή πεπερασμένων καταστάσεων (fsm). Αυτή θα αποτελείται από πέντε states, τα οποία θα καλύπτουν τα στάδια μετάδοσης: IDLE START DATA PARITY STOP. Αφού το bit που θα στέλνουμε εξαρτάται μόνο από το στάδιο της αποστολής στο οποίο θα είμαστε, είναι λογικό η fsm που θα υλοποιήσουμε να είναι τύπου Moore (προφανώς το TxD θα είναι έξοδος της fsm). Η κύρια λογική της fsm υλοποιείται με δύο always blocks, ένα συνδιαστικής λογικής, που θα θέτει τις εξόδους καθώς και το επόμενο state που θα πρέπει να πάρει, καθώς και ένα ακολουθιακής λογικής, που θα αναθέτει ανα κύκλο του ρολογιού το επόμενο state σαν το current state.

Θα αρχίσουμε να στέλνουμε δεδομένα όταν ανεβεί το σήμα Tx_WR, αρκεί να είναι ανοιχτός, δηλαδή αρκεί το Tx_EN να είναι ίσο με ένα. Μέχρι να ισχύει η συνθήκη $Tx_EN = 1 \ \&\& \ Tx_WR = 1$ ο αποστολέας θα είναι ανενεργός, δηλαδή στο idle state. Όταν είναι ανενεργός θα στέλνει σταθερό 1, ενώ το start bit που θα αποσταλεί θα είναι 0 και το stop bit 1.

Ο αποστολέας θα στέλνει νέο δεδομένο κάθε δεκαέξι παλμούς του Tx_sample_ENABLE (που είναι το sampling σήμα που θα έχει σαν έξοδο του το baud_controller). Για να το επιτύχουμε αυτό θα υλοποιήσουμε σε μία νέα always block ακολουθιακής λογικής έναν μετρητή 4bit, ο οποίος θα αυξάνεται με κάθε νέο sampling σήμα που θα λάβουμε. Όταν ο μετρητής γίνει ίσος με δεκάξι, θα ανοίγει ένα νέο σήμα switch για ένα κύκλο, το οποίο θα επιτρέψει την fsm να αλλάξει κατάσταση. Αξίζει να σημειωθεί πως για να είναι ο χρόνος αποστολής του start bit ανάλογος με τον χρόνο αποστολής των υπολοίπων σημάτων, ο μετρητής θα μηδενίζεται όταν είμαστε στο idle state.

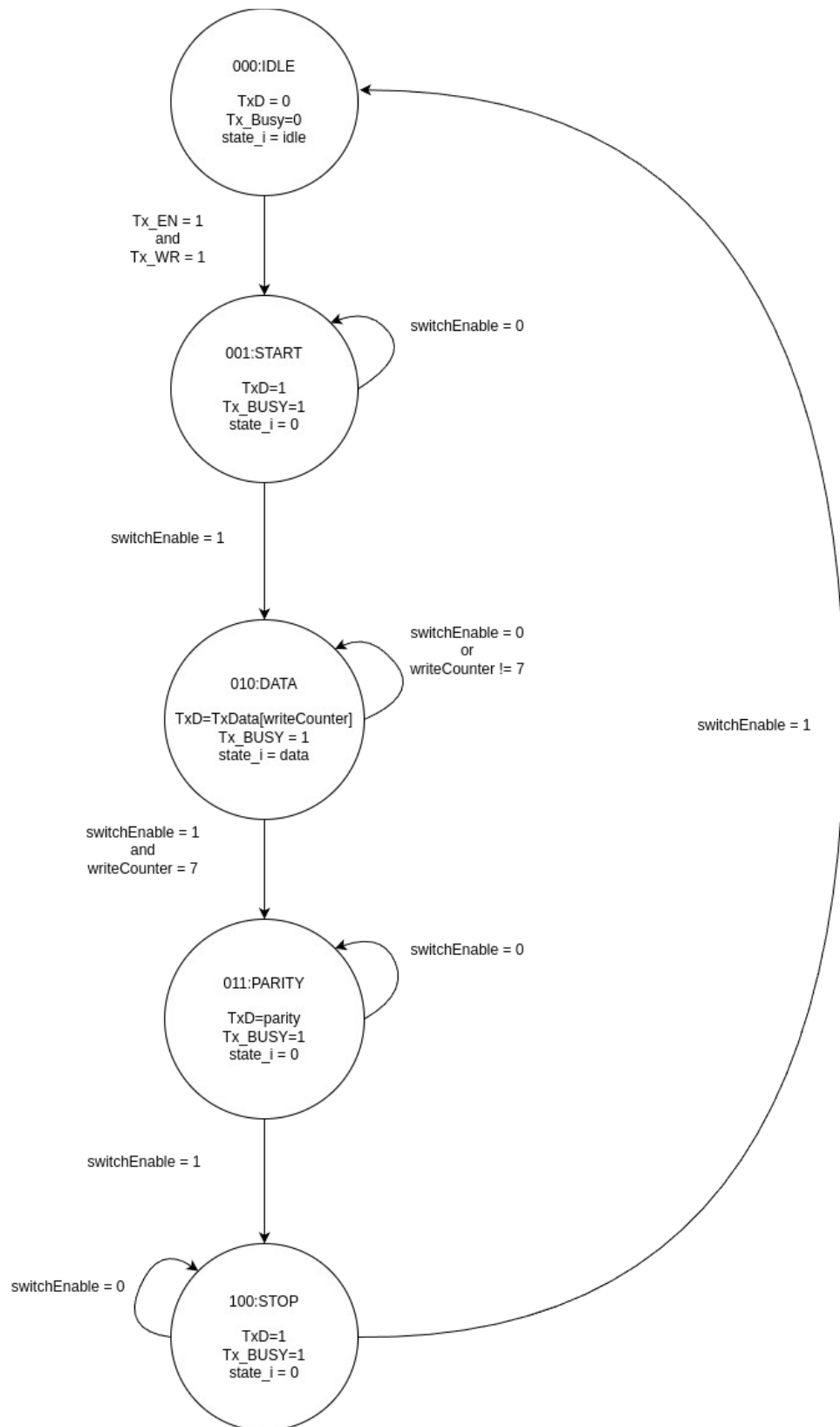
Σε αντίθεση με τα υπόλοιπα states, στο data state δεν θα στείλουμε μόνο ένα bit, αλλά όλο το περιεχόμενο του χαρακτήρα. Θα υλοποιήσουμε σε μία νέα always block ακολουθιακής λογικής έναν 3-bit μετρητή που θα προσμετράει bit που θα στείλουμε όσο είμαστε στο data, τον οποίο θα χρησιμοποιήσουμε και στην έκφραση $TxD = Tx_DATA[writeCounter]$ (writeCounter ο μετρητής). Ο μετρητής θα αυξάνεται μόνο όσο βρισκόμαστε στο data state.

Θα χρειαστεί επίσης να υπολογίσουμε το parity bit, κάτι το οποίο υλοποιείται με ένα xor reduction του Tx_DATA, επίσης σε μία νέα always block ακολουθιακής λογικής (χρησιμοποιούμε even parity). Για λόγους ευκολίας θα το υπολογίσουμε όταν είμαστε στο data state.

Τέλος στο stop state θα στέλνουμε απλώς το bit εξόδου, δηλαδή εδώ 1. Μετά την αποστολή του bit θα έχει τελειώσει τη μετάδοση ο αποστολέας οπότε θα επιστρέψει στο idle

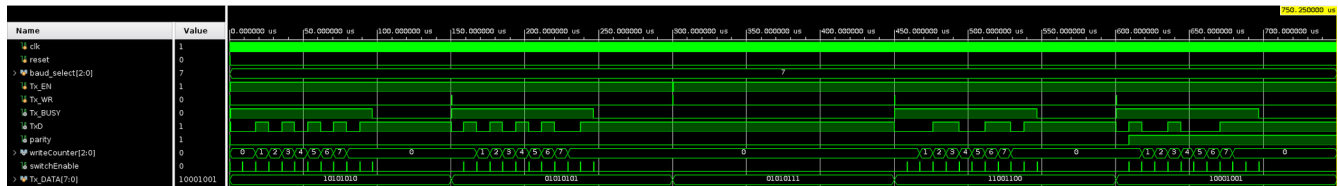
Είσοδοι της fsm θα είναι τα switchEnable, writeCounter, parity, Tx_EN, Tx_WR, Tx_DATA. Σαν εξόδους, πέρα από το TxD και το Tx_BUSY, θα έχουμε και δύο καταχωρητές, τα οποία θα σηματοδοτούν αν είμαστε στο idle ή στο data state, για να μπορούν να λειτουργούν οι μετρητές και το parity σωστά.

Παρακάτω φαίνεται ένα διάγραμμα λειτουργίας της fsm. Εξυπακούεται πως η έξοδος state_i είναι συμβολική (για αναγνωσιμότητα), και οι πραγματικοί εξόδοι είναι οι καταχωρητές που περιγράφονται παραπάνω (υπάρχουν όταν $state_i \neq 0$ στο σχήμα). Το ίδιο θα ισχύει και στο 3ο μέρος της εργασίας



PartB Fsm Diagramm

Για την επαλήθευση θα δοκιμάσουμε να στείλουμε πέντε μηνύματα, εκ των οποίων τα τέσσερα θα παραδωθούν κανονικά (είναι αυτά που δώθηκαν στην εκφώνηση ως παράδειγμα), ενώ το ένα δεν θα αποσταλλεί καθόλου επειδή το Tx_EN θα είναι ίσο με 0. Θέτουμε το baud select = 7 για να το δοκιμάσουμε όσο το δυνατό περισσότερο στο μικρότερο χρονικό διάστημα (το ίδιο θα κάνουμε και στα δύο επόμενα κομμάτια της εργασίας).



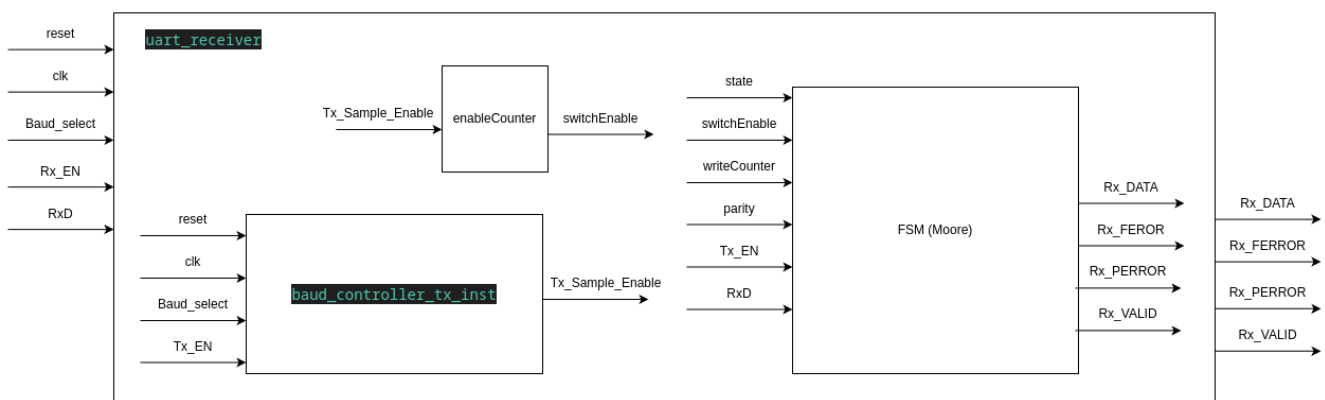
PartB UART Transmitter Post-Synthesis Functional Simulation

Βλέπουμε ότι στο παραπάνω ότι όντως τα μηνύματα στέλνονται κανονικά, με εξαίρεση το μεσάιο που έχουμε κλείσει το Tx_EN. Στο τελευταίο μήνυμα φαίνεται και η σωστή ανάθεση του parity. Βλέπουμε επίσης τη σωστή ανάθεση του TxD και του Tx_DATA.

• Μέρος 3ο: Υλοποίηση UART δέκτη (receiver)

Σε αυτό το κομμάτι της εργασίας υλοποιούμε έναν UART δέκτη, που θα μπορεί να αποκωδικοποιήσει τα μηνύματα που θα στέλνει ο αποστολέας παραπάνω. Αυτή τη φορά θα έχουμε ένα bit σαν είσοδο, από το οποίο θα ανακατασκευάσουμε έναν 8bit χαρακτήρα. Θα έχουμε πάλι στις εισόδους το Rx_EN και το baud_select, το οποίο θα πρέπει να έχει την ίδια τιμή με αυτό του αποστολέα. Πέρα από το Rx_DATA, ο δέκτης επιστρέφει και κάποια σήματα ελέγχου σχετικά με τη ποιότητα των δεδομένων που λάβαμε, κοινώς τα Rx_FERROR, Rx_PERROR, Rx_VALID που υποδυκνείουν λάθος στο framing, λάθος στο parity και σωστή καταγραφή αντίστοιχα.

Παρακάτω φαίνεται ένα flow diagramm για τον δέκτη. Λεπτομέρειες στην υλοποίηση εξηγούνται αναλυτικά μετά το διάγραμμα. Αξίζει να σημειωθεί πως το block της fsm περιλαμβάνει και ένα always block όπως θα εξηγηθεί και παρακάτω. Αυτό δεν είναι κομμάτι της πραγματικής fsm, το διάγραμμα όμως αφήνεται ως έχει για ευκολία στην αναγνωσιμότητα.



PartC Dataflow Diagramm

Προφανώς θα χρησιμοποιήσουμε και εδώ μια μονάδα `baud_controller`, για να ευθυγραμμίσουμε σωστά τη δειγματοληψία που θα κάνει ο δέκτης στα αποσταλμένα δεδομένα.

Θα χρησιμοποιήσουμε και εδώ μια `fsm`, αλλά αυτή τη φορά θα προσθέσουμε τρία επιπλέον `states` από αυτά που θα έχουμε παραπάνω: το `VALID`, το `FERROR` και το `PERROR`, από τα οποία θα εξαρτηθεί η ανάθεση των σημάτων ελέγχου που αναφέρθηκαν προηγουμένως. Αφού η αποστολή γίνεται σε στάδια, στέκει πως και για τον δέκτη, οι εξόδοι του `fsm` θα εξαρτώνται από το στάδιο στο οποίο βρισκόμαστε, δηλαδή είναι και πάλι “βολικό” το `fsm` που θα υλοποιήσουμε να είναι τύπου `moore`. Η λογική του υλοποιείται όμοια με πριν.

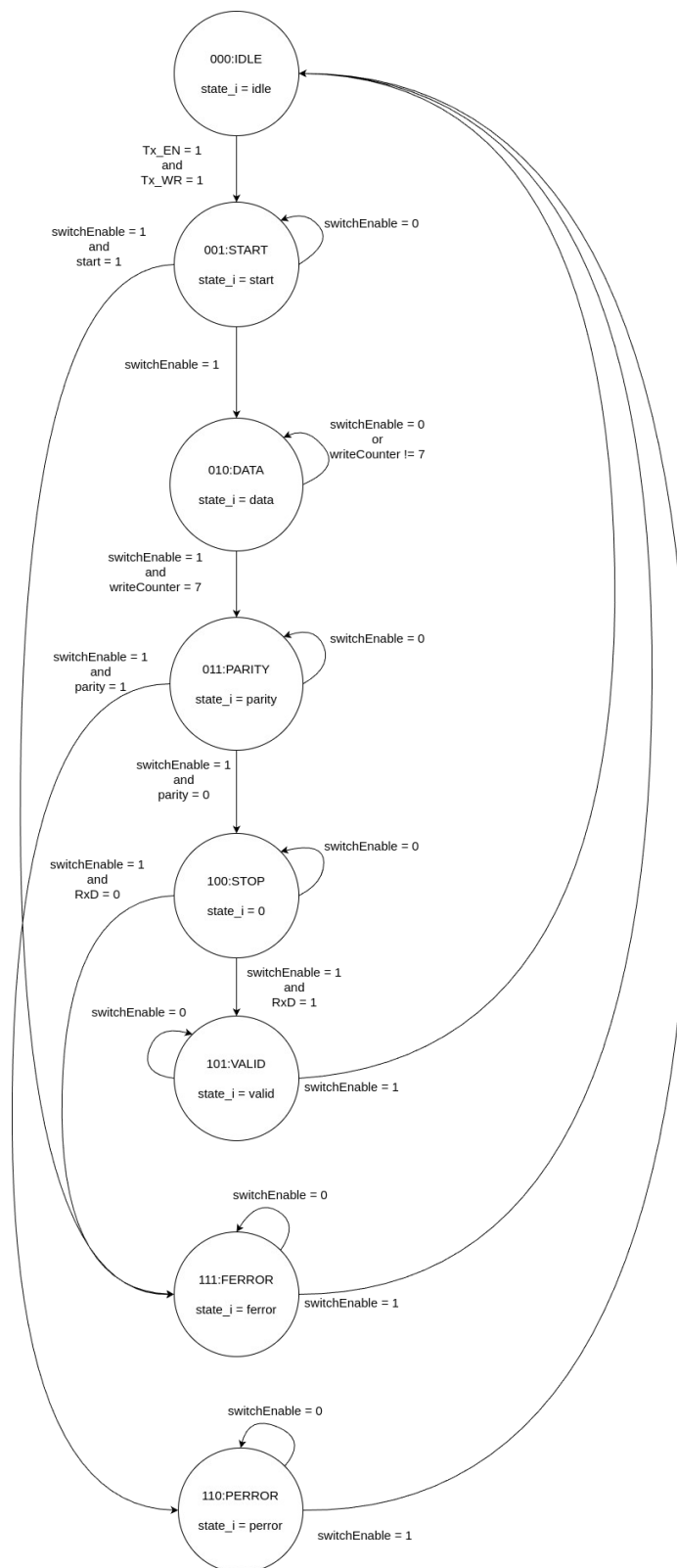
Επειδή επιθυμούμε να κρατήσουμε τα δεδομένα των εξόδων κατά τη διάρκεια πολλών διαφορετικών κύκλων, δεν θα τις χρησιμοποιήσουμε σαν εξόδους της `fsm` απευθείας, αλλά θα χρησιμοποιήσουμε μια `always block` ακολουθιακής λογικής, στην οποία θα γίνονται αναθέσεις ανάλογα με το `state` στο οποίο βρισκόμαστε. Οι εξόδοι της `fsm` δηλαδή θα είναι εφτά καταχωρητές, ο καθένας από αυτούς θα παίρνει τη τιμή 1 μόνο για ένα `state` (π.χ. ένας καταχωρητής `idle_i` θα γίνεται 1 μόνο όταν είμαστε στο `idle state`). Θα καλύπτουν έτσι όλα τα `states` εκτός από το `stop`, για το οποίο δεν μας ενδιαφέρει να αναθέσουμε κάτι.

Θα μας χρειαστούν οι `always blocks` που χρησιμοποιήσαμε προηγουμένως, με μερικές αλλαγές. Το σήμα `switchEnable` θα ανεβαίνει πλέον (για έναν κύκλο) στη μέση του μετρητή των `sampling` σημάτων και όχι όταν μεγιστοποιηθεί η τιμή του, επειδή η δειγματοληψία στη μέση της αποστολής είναι πιο σταθερή. Επίσης το `parity` το υπολογίζουμε με την έκφραση `parity <= ^Rx_DATA_buf + RxD` στο στάδιο `parity`, όπου έχουμε όλα τα δεδομένα που μας αφορούν.

Η `always` που ευθύνεται για τις αναθέσεις εξόδων θα χρησιμοποιεί έναν `buffer` πριν αναθέσει τα δεδομένα στον `Rx_DATA` (τον `Rx_DATA_buf`), για να μπορεί να κρατήσει τα προηγούμενα σε περίπτωση λάθους. Αυτός και τα σήματα ελέγχου μηδενίζονται όταν είμαστε στο `start state`. Όσο είμαστε στο `data state`, θα έχουμε την έκφραση `Rx_DATA_buf[writeCounter] <= RxD`, όπου ο `writeCounter` θα έχει την ίδια λογική με αυτόν που χρησιμοποιήθηκε στον `transmitter`, όπως αναφέρθηκε πριν. Θα αναθέσουμε στα σήματα `Rx_VALID`, `Rx_FERROR` και `Rx_PERROR` την τιμή 1 όταν είμαστε στο `valid`, `error` και `perror state` αντίστοιχα. Όταν είμαστε στο `valid state`, θα αναθέσουμε τα δεδομένα του `buffer` στο `Rx_DATA`.

Για να ελέγξουμε για `framing errors`, κοιτάμε αν το `start bit` είναι όντως 0 όταν προχωράμε στην δειγματοληψία των δεδομένων, και αν το `stop bit` είναι όντως 1 στο τέλος. Αν κάτι από τα δύο δεν ισχύει, μεταβιβάζομαστε στο `FERROR state`. Για να ελέγξουμε για `parity errors`, ελέγχουμε αν η τιμή του `parity` είναι όντως ίση με 0 (χρησιμοποιούμε `even parity`). Αν το παραπάνω δεν ισχύει μεταβιβάζομαστε στο `PERROR state`. Αν τίποτα από τα δύο δεν ισχύει μεταβιβάζομαστε στο `VALID state` μετά από το `STOP state`. Αν βρεθούμε σε `error` κατάσταση π.χ. λόγω λάθος `start bit`, θα επιστρέψουμε απευθείας στο `idle state`, παρόλο που ο αποστολέας μπορεί να συνεχίζει να στέλνει δεδομένα. Είναι ευθύνη οποιουδήποτε χρήστη δηλαδή να κλείσει τον δέκτη μέχρι ο αποστολέας να σταματήσει τη μετάδοση, σε περίπτωση λάθους.

Όπως τα `START`, `DATA`, `PARITY` και `STOP states` στον αποστολέα ήταν υπεύθυνα για την αποστολή των δεδομένων όπως προβλέπει το `UART` πρωτόκολλο, έτσι και εδώ θα είναι υπεύθυνα για τη λήψη τους (στο μέσο της αποστολής), όπως και εξηγήθηκε πιο λεπτομερώς παραπάνω. Εδώ η μεταβίβαση από το `idle` στο `start state` θα γίνει αν ο δέκτης είναι ενεργοποιημένος και λάβει το `start bit`, δηλαδή όταν είναι αληθής η έκφραση `RxD == 0 && Rx_EN == 1`. Προφανώς εδώ τα `START`, `STOP` και `PARITY states` μπορεί να μεταβιβάσουν την `fsm` σε `error state`, και η `stop` θα μεταβιβάσει την `fsm` στο `valid state` και όχι απευθείας στο `idle`. Αυτό πλέον το κάνουν το `valid` και τα `error states`.



PartC Fsm Diagramm

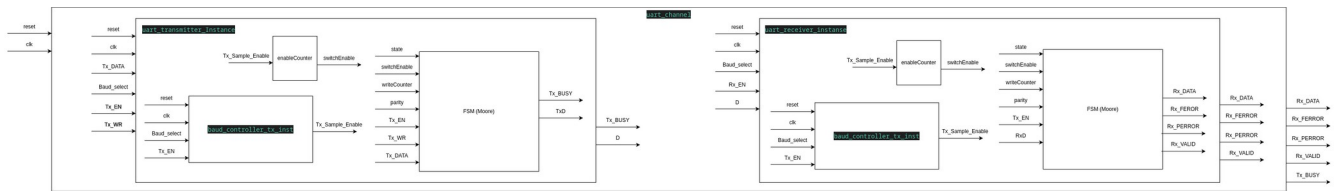
Name	Value	0.000000 us	100.000000 us	200.000000 us	300.000000 us	400.000000 us	500.000000 us	600.000000 us	700.000000 us	800.000000 us	900.000000 us	1.000.000000 us	1.100.000000 us	
clk	0													
reset	0													
baud_select[2:0]	7													
Rs_S0	1													
Rs_P0	0													
Rs_DATA[7:0]	aa													
Rs_ERROR	0													
Rr_ERROR	1													
Rr_VALID	0													
writesCounter[2:0]	0													
switch(enable)	0													
counter[4:0]	0000a													

PartC UART Receiver Post-Synthesis Functional Simulation

Απο τις παραπάνω κυμματομορφές, όντως επιβεβαιώνεται η ορθή λειτουργία του δέκτη. Η έξοδος όντως αλλάζει μόνο αφού επιβεβαιωθεί ότι το μήνυμα που στάλθηκε είναι σωστό, λόγω του buffer. Τα σήματα σφαλμάτων είναι όντως όπως τα περιμέναμε.

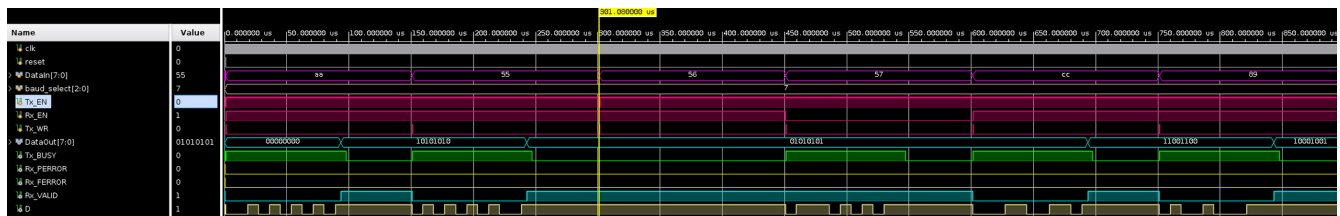
- **Μέρος 4ο: Υλοποίηση UART Αποστολέα-Δέκτη για Σειριακή Μεταφορά Δεδομένων**

Σε αυτό το κομμάτι της εργασίας υλοποιούμε ένα top module στο οποίο θα κάνει intantiate έναν UART αποστολέα, καθώς και έναν UART δέκτη, και συνδέει τις μονάδες μεταξύ τους με ένα καλώδιο. Απο τη στιγμή που δεν θα χρειαστεί να ελέγξουμε τον κώδικα στη πλακέτα, μπορούμε να παραλείψουμε τη σύνδεση του reset σε synchroniser και anti-bounce μονάδες για ευκολία. Μπορούμε επίσης να έχουμε τις εισόδους του αποστολέα και του δέκτη σαν εισόδους στο top, και τις εξόδους τους σαν εξόδους του αντίστοιχα, για καλύτερο έλεγχο στο testbench. Παρακάτω φαίνεται το flow diagramm



PartD Dataflow Diagramm

Για την επαλήθευση, στέλνουμε με τον αποστολέα έξι μηνύματα, 4 απο τα οποία θα σταλθούν κανονικά, ενώ δύο απο αυτά θα χαθούν γιατί θα είναι κλειστός ο αποστολέας και ο δέκτης αντίστοιχα. Δεν μπορούμε να ελέγχουμε για πιθανά σφάλματα, αφού ξέρουμε πως ο αποστολέας παραδίδει τα μηνύματα σωστά.



PartD Top Module Post-Synthesis Functional Simulation

Αξίζει να σημειωθεί πως σε κανονική χρήση, θα ήταν πιο σωστό η αποστολή (ή μη) του επόμενου μηνύματος να εξαρτάται απο τα σήματα ελέγχου που είναι έξοδοι της μονάδας αντί να στέλνονται μετά απο σταθερή καθυστέρηση, αλλά στο πλαίσιο του ελέγχου της ορθής λειτουργίας της μονάδας η σταθερή καθυστέρηση αρκεί.

• Συμπεράσματα:

Σε αυτή την εργασία υλοποιήσαμε αρχικά έναν uart αποστολέα και έπειτα έναν uart δέκτη. Κατα την πορεία του εργαστηρίου χρειάστηκε να αλλάξουν ελαφρώς, κυρίως για τη σωστή λειτουργία της fsm, και την αποτροπή κακών πρακτικών όπως την μή ανάθεση όλων των εξόδων σε κάθε state ή την εξάρτηση λογικής εκτός της fsm απο το state, ή λόγω απροσεξίας κατα την ανάγνωση της εκφώνησης. Η δοκιμές διεξάχθηκαν χωρίς ιδιαίτερες δυσκολίες, με εξαίρεση μερικές διορθώσεις που έπρεπε να γίνουν στα testbench για να αρχικοποιούνται σωστά οι εισόδοι πριν τη δοκιμή. Κατα τη σύνθεση φάνηκαν λάθη τα οποία όμως ήταν απλά. Αξίζει να σημειωθεί πως ο συνθέτης βγάζει 3 warning σε όλα τα μέρη της εργασίας τα οποία αφορούν τη παράλληλη σύνθεση και το HSR_BOUNDARY_TOP, που μπορούν να αγνοηθούν.